

VulnLab: A Reproducible Platform for Generating Vulnerable-by-Design Android Training Labs

Mohamed Bril, Youssef Charaf, Mouad Charraj, Ibtissam El Bekkali
Cadi Ayyad University, National School of Applied Sciences,
Cyber-Defense Engineering and Telecommunication Systems, Marrakech, Morocco
Supervised by Prof. M. Lachgar

Abstract

Mobile security training needs practical and safe environments where students can study Android vulnerabilities without attacking real systems. VulnLab is a Docker-based web platform that generates vulnerable-by-design Android training labs for classroom use. The platform allows an instructor to select a vulnerability module, generate a practical challenge, export the lab in Markdown or JSON, build vulnerable and patched APK files, and follow student submissions through a CTF-style workflow. VulnLab uses a React 18/Vite frontend, a Node.js/Express REST API, local JSON storage, and an Android build chain integrated into the backend. The current version contains eight pedagogical modules covering beginner, intermediate, and advanced mobile security topics. The platform also supports progressive hints, quiz questions, flags, student write-ups, and teacher feedback.

Keywords: mobile security, Android, vulnerable application, cybersecurity training, CTF, Docker, OWASP MASVS

Metadata

Nr.	Metadata description	Information
C1	Current code version	v1.0.0
C2	Permanent link to code/repository used for this code version	https://github.com/Jettleee/apk_vulnerable_generator
C3	Permanent link to Reproducible Capsule	https://www.youtube.com/watch?v=fKVrczlnqhc
C4	Legal Code License	MIT License
C5	Code versioning system used	Git
C6	Software code languages, tools, and services used	JavaScript, React 18, Vite, TailwindCSS, React Router, Node.js, Express, Docker, Docker Compose, JDK 17, Android SDK, aapt2, d8, zipalign, apksigner
C7	Compilation requirements, operating environments & dependencies	Docker and Docker Compose. The frontend runs on port 3000 and the backend runs on port 3001.
C8	Link to developer documentation/-manual	https://github.com/Jettleee/apk_vulnerable_generator/blob/main/README.md
C9	Support email for questions	charrajmouad@gmail.com

Table 1: VulnLab project metadata

1. Motivation and significance

Mobile application security is an important topic in cybersecurity education. Android applications can contain exposed components, hardcoded secrets, insecure storage, weak input validation, sensitive logs, debug configurations, or insecure network settings. Students need controlled exercises where they can observe these issues, reproduce them, and learn how to correct them without using real targets. This is consistent with the educational use of the OWASP Mobile Application Security Verification Standard and the OWASP Mobile Application Security Testing Guide [1, 2].

In many practical sessions, teachers prepare APK files manually, write the scenario in a separate document, create quiz questions in another tool, and evaluate the report manually. This process is repetitive and can lead to differences between groups. It also makes reproducibility more difficult because the APK, the statement, the hints, the correction, and the evaluation are not produced from the same source.

VulnLab addresses this problem by generating a complete Android security lab from structured vulnerability modules. Each module contains a learning objective, a risk explanation, a vulnerable example, a patched version, progressive hints, quiz questions, a flag, and teacher correction material. The

platform therefore combines content generation, Android APK construction, and student submission tracking in one workflow.

The goal of VulnLab is strictly educational. It does not provide an offensive platform for attacking real systems. It creates small Android applications with fictitious data, designed for authorized classroom practice and self-learning in an isolated environment.

2. Software description

VulnLab is a web platform for generating and managing Android security training labs. The teacher selects a vulnerability module and the system produces a structured challenge. Students receive a challenge link, download the APK, analyze it, use hints if needed, answer the quiz, submit the flag, and write a short technical report. The teacher can then review the submission and add feedback.

2.1. Software Architecture

The VulnLab architecture is designed to keep the teacher workflow, student workflow, backend services, local storage, and Android build process connected but separated. The main components are shown in Figure 1.

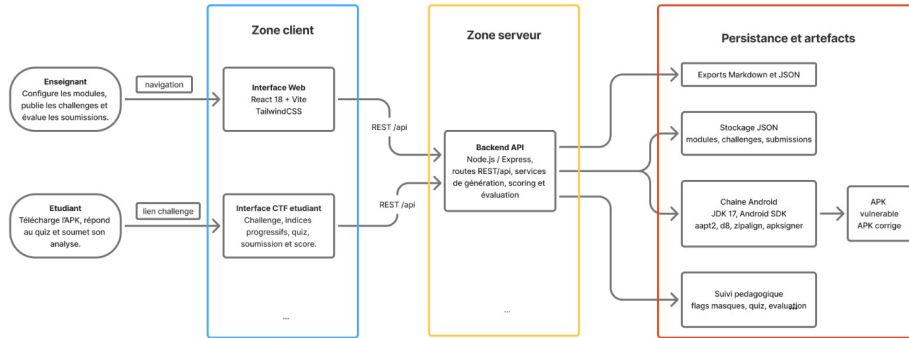


Figure 1: VulnLab technical architecture and main flows between the teacher, the student, the API, JSON storage, and the Android build chain.

1. Overview of the Architecture

VulnLab follows a client-server architecture. The client side contains the teacher interface and the student challenge interface. The server side contains the REST API, generation services, scoring logic, export services, and

Android build services. The persistence part stores modules, challenges, and submissions in local JSON files. The Android build chain produces vulnerable and patched APK files. Docker and Docker Compose are used to make the environment easier to reproduce [4].

2. Backend Infrastructure

The backend is implemented with Node.js and Express [5, 6]. It exposes REST endpoints for health checks, module listing, lab generation, Markdown export, APK building, challenge creation, student submission, and teacher evaluation. The backend coordinates the whole lab generation process and connects the pedagogical content with the Android build chain.

Table 2: Main VulnLab API routes.

Method	Route	Role
GET	/api/health	Check backend availability.
GET	/api/modules	List the available pedagogical modules.
GET	/api/modules/:id	Get the details of one module.
POST	/api/generate	Generate a complete lab scenario.
POST	/api/export/markdown	Export the lab as Markdown.
POST	/api/apk/build	Build a vulnerable or patched APK.
GET/POST	/api/challenges	Create and consult challenges.
POST	/api/challenges/:id/submit	Receive a student flag, quiz answers, and write-up.
POST	/api/challenges/:id/evaluate	Save the teacher score and feedback.

3. Frontend Composition

The frontend is implemented with React 18, Vite, TailwindCSS, and React Router [7, 8, 9]. It contains two main user paths. The teacher path is used to generate challenges, copy student links, monitor submissions, and provide feedback. The student path is used to read the challenge, download the APK, reveal hints, answer the quiz, submit the flag, and write the report.

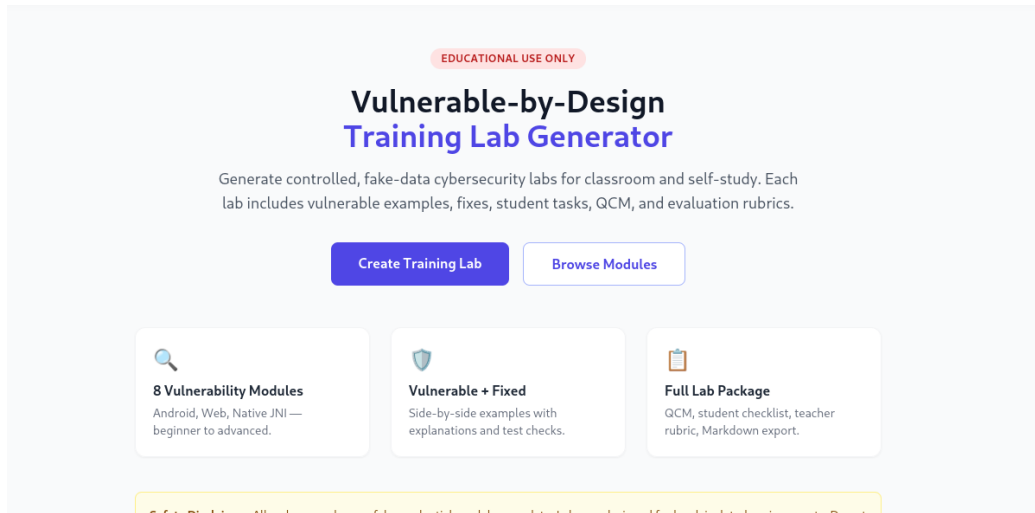


Figure 2: VulnLab home page presenting the educational purpose of the platform.

4. Data Management

The current version uses local JSON files to store modules, generated challenges, and submissions. This choice keeps installation simple and is suitable for a classroom prototype. However, JSON storage is not ideal for a large multi-user deployment because it does not provide strong concurrency control, role-based access management, or advanced audit history. A future version should migrate this layer to a relational database such as PostgreSQL or MySQL.

5. Android Build Chain

The Android build chain is integrated into the backend environment. It uses JDK 17, Android SDK tools, `aapt2`, `d8`, `zipalign`, and `apksigner`. This design avoids manual tool installation on each machine and makes the build process more reproducible. The platform can generate a vulnerable APK and, when the module provides it, a patched APK for comparison. This is important because students can see both the unsafe behavior and the mitigation.

6. Communication and Data Flow

The teacher starts by selecting a module and challenge options. The frontend sends the request to the backend through the REST API. The backend loads the module data, generates the learning content, prepares the quiz and hints, and builds the APK if requested. The student then uses the challenge link to access the lab. After analysis, the student sends the flag, quiz answers, and write-up. The teacher reviews the submission from the admin dashboard.

7. Security and Educational Boundaries

The generated labs use fictitious data and must be used only in authorized environments. The purpose is to teach Android security concepts such as exported components, hardcoded secrets, cleartext traffic, sensitive logs, insecure file permissions, weak input validation, debug mode, and unsafe client-side native checks. The modules are aligned with common references such as OWASP MASVS, OWASP MASTG, and CWE entries [1, 2, 10, 11, 12, 13, 14, 15, 16].

2.2. Software functionalities

The platform provides a complete training workflow for instructors and students:

1. **Challenge generation:** the instructor selects a vulnerability module and creates a lab with objective, risk description, hints, quiz, flag, and correction material.
2. **APK production:** the platform builds vulnerable APK files and patched APK files when a corrected version is available.
3. **Markdown and JSON exports:** the lab can be exported as Markdown for course material and as JSON for reuse by other tools.
4. **Student CTF workflow:** the student downloads the APK, analyzes it, uses hints, answers the quiz, submits the flag, and writes a short report.
5. **Teacher monitoring:** the instructor follows submissions, reads write-ups, checks scores, and adds feedback.
6. **Module catalog:** the current version contains eight modules from beginner to advanced level.

Table 3: VulnLab pedagogical modules.

Module	Level	Reference	Learning objective
Exported Component	Beginner	OWASP MASVS, MASTG, CWE-926	Understand the risk of an Android component exposed without access control.
Hardcoded Secret	Beginner	MASVS-STORAGE, CWE-798	Identify a hardcoded secret and propose a correction.
Cleartext Network Config	Beginner	MASVS-NETWORK, Android security practices	Understand the impact of unencrypted HTTP traffic.
Sensitive Data in Logs	Beginner	MASVS-PRIVACY, CWE-532	Detect sensitive data written to logs.
Insecure File Permission	Intermediate	MASVS-STORAGE, CWE-732	Verify file permissions and local storage risks.
Weak Input Validation	Intermediate	CWE-20, secure coding	Evaluate insufficient client-side validation.
Insecure Debug Mode	Intermediate	Android security, CWE-489	Understand the risk of shipping an application in debug mode.
Vulnerable JNI Native Check	Advanced	MASTG, CWE-693	Study the limits of native client-side checks.

2.3. Challenge and Evaluation Process

The evaluation process combines practical exploitation and technical explanation. A valid flag confirms that the student reached the expected behavior. The quiz checks technical understanding. The write-up asks the student to explain the vulnerability, reproduction steps, impact, and correction. This structure helps the teacher evaluate both the result and the reasoning.

3. Illustrative Examples

This section presents the main workflows implemented in VulnLab using project screenshots. The examples show how a teacher creates a challenge and how a student solves it.

3.1. Teacher Challenge Generation

The teacher dashboard allows the instructor to generate a new challenge from a selected module. The teacher can choose the difficulty and the number of quiz questions. Figure 3 shows the generation screen.

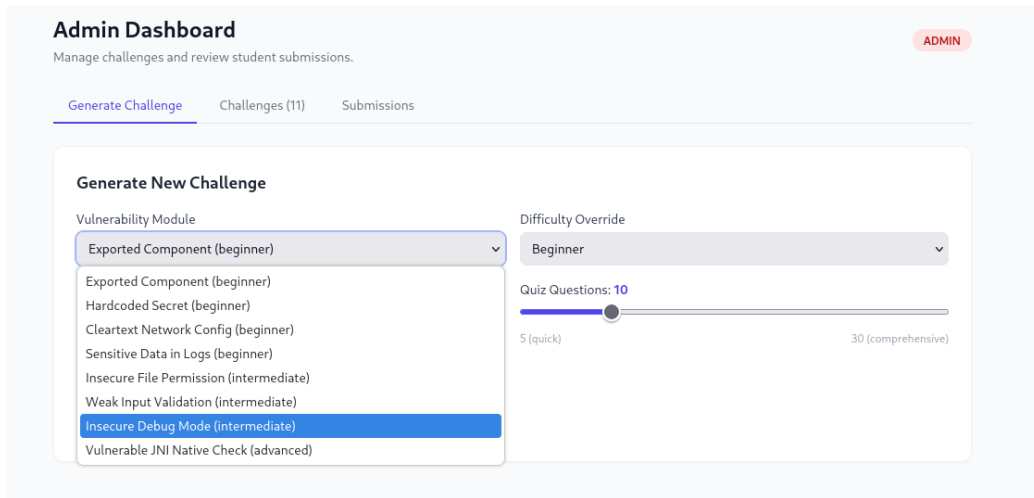


Figure 3: Teacher dashboard: generation of a new challenge from a vulnerability module.

After generation, the teacher can copy the student link and monitor submissions. Figure 4 shows the monitoring view. The flag is hidden in the public report to avoid sharing the solution outside the classroom.



Figure 4: Teacher dashboard: monitoring a generated challenge and its submissions.

3.2. Student Challenge Discovery

The student interface starts with an overview of the challenge. It presents the module, category, level, learning objective, risk overview, and challenge goal. Figure 5 shows this first step.

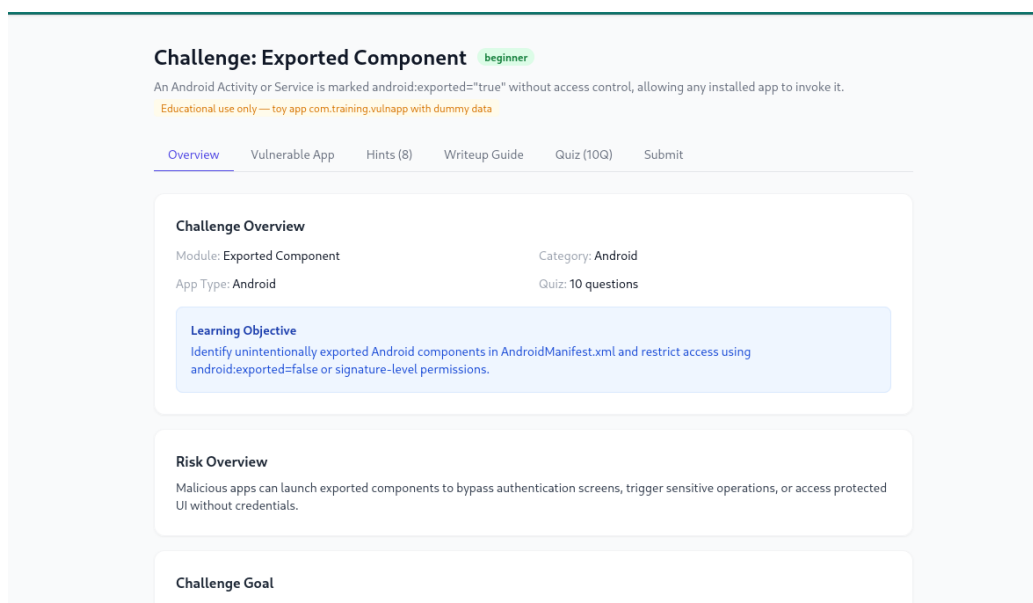


Figure 5: Student view: challenge overview and learning objective.

Progressive hints guide the student without directly giving the answer. The hints can be revealed step by step, which reduces blocking during a practical session while preserving the learning process. This is shown in Figure 6.

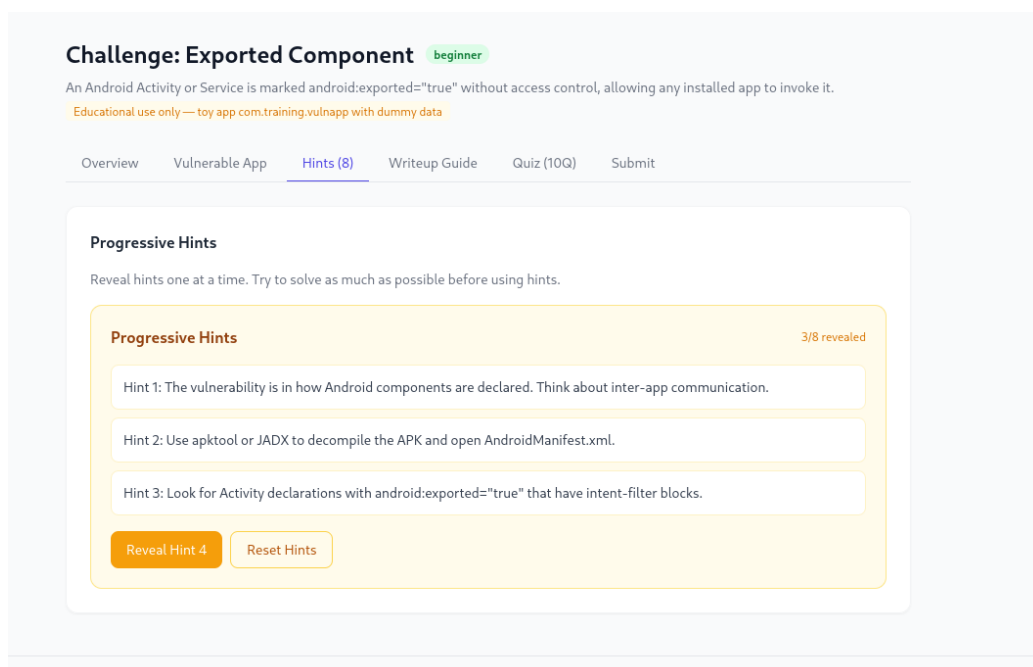


Figure 6: Student view: progressive hints for guided analysis.

3.3. Quiz, Submission, and Results

The quiz checks whether the student understood the technical details of the challenge. It may include Android configuration, ADB commands, exported components, or mitigation concepts. Figure 7 shows the quiz interface.

Challenge: Exported Component beginner

An Android Activity or Service is marked `android:exported="true"` without access control, allowing any installed app to invoke it.

Educational use only — toy app `com.training.vulnapp` with dummy data

Overview Vulnerable App Hints (8) Writeup Guide **Quiz (10Q)** Submit

Interactive Quiz (10 questions)

Q1 Multiple-Choice intermediate

Which adb command successfully launches `AdminActivity` of `com.training.vulnapp`?

adb shell · am start

☐ A. adb shell pm start com.training.vulnapp/.AdminActivity

☐ B. adb shell am start -n com.training.vulnapp/.AdminActivity

☐ C. adb launch com.training.vulnapp.AdminActivity

☐ D. adb shell intent com.training.vulnapp/.AdminActivity

Q2 True-False intermediate

An exported `Service` allows any app to bind to it and potentially call its methods.

AndroidManifest.xml · `<service>`

Figure 7: Student view: quiz associated with the challenge.

The submission form asks for the flag, the student name, and a written analysis. The write-up structure asks the student to describe the vulnerability, reproduction steps, impact, and correction. Figure 8 shows this form.

Challenge: Exported Component beginner

An Android Activity or Service is marked `android:exported="true"` without access control, allowing any installed app to invoke it.

Educational use only — toy app `com.training.vulnapp` with dummy data

[Overview](#) [Vulnerable App](#) [Hints \(8\)](#) [Writeup Guide](#) [Quiz \(10Q\)](#) [Submit](#)

Submit Your Analysis

Complete the quiz first (Quiz tab), then submit your flag and written analysis here.

Submit Flag *

Find the flag in the vulnerable APK (format: `VULNLAB{...}`) and paste it here.

Your Name (optional)

Security Analysis Writeup *

Describe: vulnerability found, reproduction steps, impact, and your proposed fix.

```
## Vulnerability Found

## Reproduction Steps

## Impact Assessment
```

Figure 8: Student view: flag and write-up submission form.

The result screen combines flag validity, quiz answers, and write-up evaluation. This gives immediate feedback while keeping space for teacher feedback. Figure 9 shows an example result screen.

Challenge: Exported Component beginner

An Android Activity or Service is marked `android:exported="true"` without access control, allowing any installed app to invoke it.

Educational use only — toy app `com.training.vulnapp` with dummy data

Overview Vulnerable App Hints (8) Writeup Guide Quiz (10Q) Submit **Results**

10%

Final Score (flag 50% + quiz 30% + writeup 20%)

✗ Flag Incorrect (0%)

Quiz: 0 / 10 correct (0%)

Review the material and try again.

Go back to the APK, keep digging — the flag is in there!

Question Review

- ✗ Which adb command successfully launches AdminActivity of `com.training.vulnapp`?
Correct: B
am start -n uses component name format package/class to launch activities.
- ✗ An exported Service allows any app to bind to it and potentially call its methods.

Figure 9: Student view: result page and answer review.

3.4. Teacher Monitoring and Patched App Access

This part is not a separate JNI case study. It shows two general platform features using the screenshots that were available from a challenge based on the Vulnerable JNI Native Check module. In the teacher dashboard, the instructor can monitor a generated challenge, view the submitted work, check the score, and add feedback. The module shown in Figure 10 is JNI only because this was the selected challenge in the captured screen.

Generate Challenge Challenges (12) **Submissions**

Challenge: Vulnerable JNI Native Check

advanced - 10 questions - A toy Android app uses a JNI native C++ library to perform a license check against a hardcoded dummy token. The Java wrapper method is the Frida instrumentation point, and the native check is bypassable at runtime. This is a controlled educational example — not a real app.

Flag masqué pour la version publique du rapport

[View Teacher Solution](#)

[Copy Student Link](#)

Submissions (1)

Submission	Status	Score
Anonymous 5/18/2026, 4:29:32 AM	✗ Flag wrong	3% (final)

Figure 10: Teacher dashboard: monitoring a submitted challenge. The example submission is linked to the JNI module.

The student interface also contains a patched application tab. This tab is used when the teacher wants students to compare the vulnerable application with the corrected version. It helps students understand the mitigation after the analysis. Figure 11 uses the JNI challenge only as an example, because this was the available screenshot. The feature itself belongs to the general student workflow and can be used with other modules when a patched application is provided.

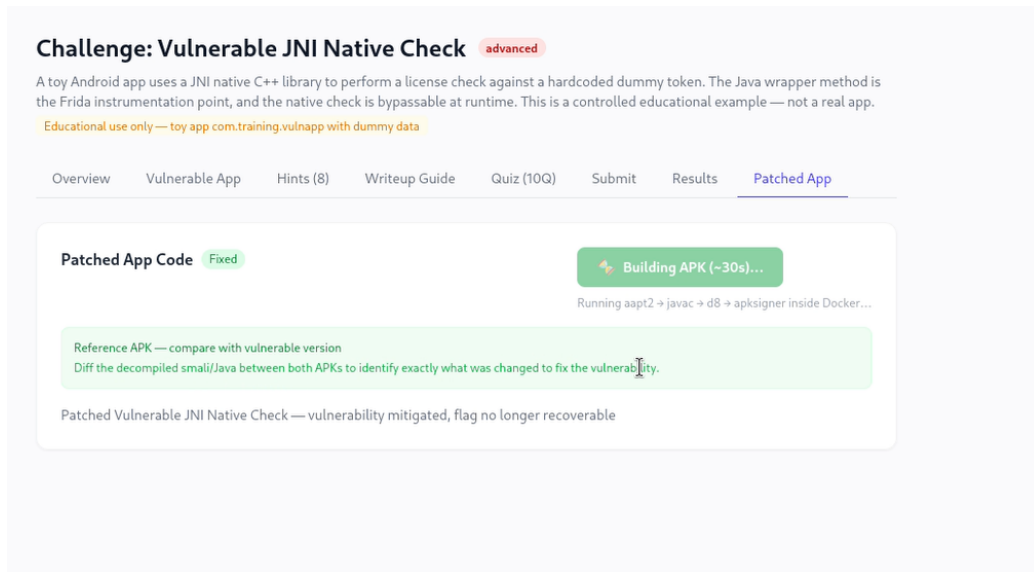


Figure 11: Student view: patched application tab used to reveal and compare the corrected APK when it is available.

4. Impact

VulnLab reduces the repetitive work needed to prepare mobile security practical sessions. The teacher does not need to prepare the scenario, quiz, hints, correction, APK, and submission tracking separately. The platform produces a coherent laboratory from one module and keeps the student workflow in the same environment.

The platform also improves evaluation traceability. A submission contains the flag, quiz answers, and a written analysis. This makes it easier to distinguish a student who only found the flag from a student who understood the vulnerability and can explain the correction.

4.1. Comparison with Existing Learning Supports

Several intentionally vulnerable Android applications are useful for training, including DIVA and InsecureBankv2 [17, 18]. However, these applications are mostly fixed targets. They do not generate a complete lab package with teacher monitoring, quiz questions, progressive hints, write-up structure, and patched APK comparison. Classic CTF platforms usually provide scoring, but the pedagogical material and teacher feedback are often managed separately. VulnLab focuses on a complete classroom workflow.

Table 4: Comparison of VulnLab with related mobile security learning supports.

Criterion	VulnLab	DIVA	InsecureBankv2	Classic CTF
Lab generation	Configurable pedagogical modules	Fixed support	Fixed support	Depends on the platform
APK variants	Vulnerable and patched APK files	Fixed APK	Fixed APK	Depends on challenge
Quiz and write-up	Integrated into the student workflow	Not integrated	Not integrated	Often separate
Teacher monitoring	Dashboard, submissions, score, and feedback	Not provided	Not provided	Generic scoring
Mobile references	OWASP MASVS, MASTG, CWE	Android exercises	Android exercises	Depends on content

5. Reproducibility and Quality Assurance

The project is reproducible through Docker and Docker Compose. The frontend is available on port 3000 and the backend on port 3001. The Android build tools are included in the backend environment, which avoids manual installation on each classroom computer.

Listing 1: Local startup commands.

```
git clone https://github.com/Jettlee/apk_vulnerable_generator.git
cd apk_vulnerable_generator
docker compose up --build
```

The verification process covers backend availability, module loading, challenge generation, Markdown export, APK building, quiz display, submission storage, and teacher evaluation. This process checks that the teacher-student workflow works from the beginning of a lab to the final feedback.

6. Limitations and Future Work

The current version is a pedagogical prototype. JSON storage is simple, but it is limited for large multi-user use. Future work should add a relational

database, role-based authentication, better challenge isolation, and stronger access control between teacher and student areas.

Another limitation is challenge diversity. The current modules provide structured content, but future versions could generate more controlled variants of the Android resources, triggering paths, and flag placement. This would reduce solution reuse between groups while keeping the exercises verifiable by the teacher.

Future work can also improve quality assurance by adding unit tests for back-end routes, integration tests for the teacher-student workflow, and automated checks for APK generation. A frozen release archive and a DOI would also improve reproducibility for academic publication.

7. Conclusions

VulnLab provides a reproducible platform for generating vulnerable-by-design Android labs in a controlled educational context. It combines scenario generation, APK construction, Markdown and JSON exports, progressive hints, quiz questions, flags, student write-ups, and teacher monitoring. The platform is useful for mobile security training because it keeps the practical exercise, assessment, and correction inside one workflow.

The prototype can be improved with stronger persistence, complete authentication, automated tests, frozen releases, and a reproducible archive. These improvements would make VulnLab more robust while preserving its main goal: helping instructors teach Android security through safe and structured practical laboratories.

References

- [1] OWASP Foundation. OWASP Mobile Application Security Verification Standard (MASVS). Available: <https://mas.owasp.org/MASVS/>.
- [2] OWASP Foundation. OWASP Mobile Application Security Testing Guide (MASTG). Available: <https://mas.owasp.org/MASTG/>.
- [3] Android Developers. Android app security best practices. Available: <https://developer.android.com/privacy-and-security/security-best-practices>.
- [4] Docker Inc. Docker documentation. Available: <https://docs.docker.com/>.

- [5] OpenJS Foundation. Node.js documentation. Available: <https://nodejs.org/en/docs>.
- [6] OpenJS Foundation. Express documentation. Available: <https://expressjs.com/>.
- [7] Meta Open Source. React documentation. Available: <https://react.dev/>.
- [8] Vite Team. Vite documentation. Available: <https://vite.dev/>.
- [9] Tailwind Labs. Tailwind CSS documentation. Available: <https://tailwindcss.com/docs>.
- [10] MITRE. CWE-926: Improper Export of Android Application Components. Available: <https://cwe.mitre.org/data/definitions/926.html>.
- [11] MITRE. CWE-798: Use of Hard-coded Credentials. Available: <https://cwe.mitre.org/data/definitions/798.html>.
- [12] MITRE. CWE-532: Insertion of Sensitive Information into Log File. Available: <https://cwe.mitre.org/data/definitions/532.html>.
- [13] MITRE. CWE-732: Incorrect Permission Assignment for Critical Resource. Available: <https://cwe.mitre.org/data/definitions/732.html>.
- [14] MITRE. CWE-20: Improper Input Validation. Available: <https://cwe.mitre.org/data/definitions/20.html>.
- [15] MITRE. CWE-489: Active Debug Code. Available: <https://cwe.mitre.org/data/definitions/489.html>.
- [16] MITRE. CWE-693: Protection Mechanism Failure. Available: <https://cwe.mitre.org/data/definitions/693.html>.
- [17] Payatu. DIVA: Damn Insecure and Vulnerable App for Android. Available: <https://github.com/payatu/diva-android>.
- [18] Dinesh Shetty. InsecureBankv2: vulnerable Android application for security training. Available: <https://github.com/dineshshetty/Android-InsecureBankv2>.